

SQL Practice Set of Questions:

Create Tables

```
CREATE TABLE Department
(
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);
```

```
CREATE TABLE Employee
(
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DeptID INT,
    Salary DECIMAL(10,2),
    JoiningDate DATE,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

```
CREATE TABLE Sales
(
    SaleID INT PRIMARY KEY,
    EmpID INT,
    SaleDate DATE,
    Amount DECIMAL(10,2),
    FOREIGN KEY (EmpID) REFERENCES Employee(EmpID)
);
```

Insert Sample Data

INSERT INTO Department VALUES

(1,'HR'),
(2,'IT'),
(3,'Finance');

INSERT INTO Employee VALUES

(101,'Amit',2,75000,'2022-01-15'),
(102,'Priya',2,85000,'2021-05-10'),
(103,'Rahul',1,50000,'2023-02-20'),
(104,'Sneha',3,90000,'2020-08-12'),
(105,'Karan',2,85000,'2021-11-18'),
(106,'Pooja',1,50000,'2023-04-10'),
(107,'Vikas',3,95000,'2019-07-25');

INSERT INTO Sales VALUES

(1,101,'2024-01-01',10000),
(2,101,'2024-02-01',12000),
(3,102,'2024-01-01',15000),
(4,102,'2024-02-01',18000),
(5,103,'2024-01-01',8000),
(6,104,'2024-01-01',20000),
(7,105,'2024-02-01',17000),
(8,106,'2024-02-01',9000),
(9,107,'2024-01-01',25000);

1. ROW_NUMBER()

Basic

1. Assign row numbers to all employees based on salary (highest first).
2. Assign row numbers department-wise based on salary.
3. Find the employee having Row Number = 1 in each department.
4. Display employees with row numbers ordered by joining date.
5. Find the latest joined employee using ROW_NUMBER().

Intermediate

6. Find the top 2 highest-paid employees from each department.
 7. Remove duplicate employee records using ROW_NUMBER().
 8. Find the second highest salary employee using ROW_NUMBER().
 9. Display every employee with their position in salary ranking.
 10. Find the newest employee from each department.
-

2. RANK()

Basic

1. Rank employees based on salary.
2. Rank employees within each department.
3. Show how ranks change when two employees have the same salary.
4. Find employees with Rank = 1.
5. Find Top 3 ranked employees.

Intermediate

6. Find the second highest salary using RANK().
 7. Find employees sharing the highest salary.
 8. Display salary rank department-wise.
 9. Find departments having multiple Rank 1 employees.
 10. Compare ROW_NUMBER() and RANK() outputs.
-

3. DENSE_RANK()

Basic

1. Assign dense rank to employees based on salary.
2. Dense rank employees department-wise.
3. Find employees with Dense Rank = 2.
4. Compare RANK() and DENSE_RANK().
5. Show salary levels using DENSE_RANK().

Intermediate

6. Find third highest salary using DENSE_RANK().
 7. Find all employees having second highest salary.
 8. Find top 3 salary groups.
 9. Count employees in each dense rank.
 10. Display unique salary positions.
-

4. NTILE()

Basic

1. Divide employees into 2 salary groups.
2. Divide employees into 4 salary groups.
3. Divide employees into 3 equal buckets.
4. Find employees in bucket 1.
5. Display bucket number for each employee.

Intermediate

6. Divide sales records into 4 quarters based on amount.
 7. Find highest performing bucket.
 8. Assign grades A/B/C/D using NTILE().
 9. Divide employees department-wise into 3 groups.
 10. Identify top 25% employees.
-

5. LAG()

Basic

1. Show current sale amount and previous sale amount.
2. Compare current sale with previous sale.
3. Find difference between current and previous sales.
4. Show previous employee salary.
5. Find employees whose salary is higher than previous employee.

Intermediate

6. Find month-over-month sales growth.
 7. Display previous department employee salary.
 8. Calculate percentage increase from previous sale.
 9. Find sales drops compared to previous month.
 10. Detect first sales record where growth became negative.
-

6. LEAD()

Basic

1. Show current sale and next sale.
2. Calculate difference between current and next sale.
3. Display next employee salary.
4. Find employees earning less than next employee.
5. Predict next month's sales using LEAD().

Intermediate

6. Find future sales growth.
 7. Compare current and next sale amount.
 8. Identify employees with salary immediately below the next employee.
 9. Find last sale record using LEAD().
 10. Display next joining employee.
-

7. RUNNING TOTAL

Basic

1. Calculate running total of sales.
2. Show cumulative sales per employee.
3. Calculate running salary total.
4. Running total ordered by joining date.
5. Running total department-wise.

Intermediate

6. Find cumulative monthly sales.
 7. Find first date where running total exceeded 50,000.
 8. Calculate running average along with running total.
 9. Display cumulative sales percentage.
 10. Department-wise running total of salaries.
-

8. INDEX

Theory + Practical

1. Create an index on Salary column.
2. Create a composite index on DeptID and Salary.
3. Find employees with salary greater than 80,000.
4. Why is an index useful for salary searches?
5. Which query benefits more from indexing?

Scenario-Based

6. The employee table has 10 million rows. Which columns should be indexed?
 7. Difference between Clustered and Non-Clustered Index.
 8. Difference between Primary Key and Index.
 9. When should indexes be avoided?
 10. What happens if too many indexes are created?
-

9. VIEW

Basic

1. Create a view showing employee names and departments.
2. Create a view for employees earning above 80,000.
3. Display all records from a view.
4. Modify a view.
5. Delete a view.

Intermediate

6. Create a view showing department-wise average salary.
 7. Create a view joining Employee and Sales.
 8. Update data through a view.
 9. Can aggregate views be updated?
 10. Why use views in real projects?
-

10. STORED PROCEDURE

Basic

1. Create a procedure to display all employees.
2. Create a procedure to find employees by ID.
3. Create a procedure to insert a new employee.
4. Create a procedure to update salary.
5. Execute all procedures.

Intermediate

6. Create procedure for department-wise employees.
 7. Create procedure returning employee count.
 8. Create procedure with input and output parameters.
 9. Create procedure for highest salary employee.
 10. Create procedure to delete employee.
-

11. TRIGGER

Basic

1. Create audit table for Employee.
2. Create INSERT trigger.
3. Create UPDATE trigger.
4. Create DELETE trigger.
5. Log action date whenever employee record changes.

Intermediate

6. Prevent salary from becoming negative.
 7. Store old and new salary in audit table.
 8. Restrict employee deletion.
 9. Auto-update ModifiedDate column.
 10. Count total inserts using trigger.
-

12. CTE (Common Table Expression)

Basic

1. Create a CTE displaying all employees.
2. Find employees with salary > 70,000 using CTE.
3. Find department-wise average salary using CTE.
4. Join a CTE with Department table.
5. Use multiple CTEs in one query.

Intermediate

6. Find second highest salary using CTE.
 7. Find top 3 salaries using CTE and DENSE_RANK().
 8. Find duplicate employees using CTE.
 9. Calculate running total using CTE.
 10. Create a recursive CTE generating numbers 1 to 20.
-

ANSWERS:

ROW_NUMBER()

Q1. Assign row numbers to employees based on salary (highest first)

Answer

```
SELECT EmpID,  
       EmpName,  
       Salary,  
       ROW_NUMBER() OVER(ORDER BY Salary DESC) AS RowNum  
FROM Employee;
```

Q2. Assign row numbers department-wise based on salary

Answer

```
SELECT EmpID,  
       EmpName,  
       DeptID,  
       Salary,  
       ROW_NUMBER() OVER  
       (  
         PARTITION BY DeptID  
         ORDER BY Salary DESC  
       ) AS RowNum  
FROM Employee;
```

Q3. Find highest-paid employee from each department

Answer

WITH CTE AS

```
(
    SELECT *,
        ROW_NUMBER() OVER
        (
            PARTITION BY DeptID
            ORDER BY Salary DESC
        ) AS RN
    FROM Employee
)
SELECT *
FROM CTE
WHERE RN = 1;
```

Q4. Find second highest salary employee

Answer

WITH CTE AS

```
(
    SELECT *,
        ROW_NUMBER() OVER
        (
            ORDER BY Salary DESC
        ) AS RN
    FROM Employee
)
SELECT *
FROM CTE
WHERE RN = 2;
```

Q5. Find latest joined employee

Answer

WITH CTE AS

```
(
    SELECT *,
        ROW_NUMBER() OVER
        (
            ORDER BY JoiningDate DESC
        ) AS RN
    FROM Employee
)
SELECT *
FROM CTE
WHERE RN = 1;
```

RANK()

Q1. Rank employees according to salary

Answer

```
SELECT EmpID,
    EmpName,
    Salary,
    RANK() OVER(ORDER BY Salary DESC) AS SalaryRank
FROM Employee;
```

Q2. Rank employees department-wise

Answer

```
SELECT EmpID,
    EmpName,
    DeptID,
    Salary,
    RANK() OVER
```

```
(
    PARTITION BY DeptID
    ORDER BY Salary DESC
) AS SalaryRank
FROM Employee;
```

Q3. Find employees having highest salary

Answer

WITH CTE AS

```
(
    SELECT *,
        RANK() OVER(ORDER BY Salary DESC) AS Rnk
    FROM Employee
)
SELECT *
FROM CTE
WHERE Rnk = 1;
```

Q4. Find second highest salary

Answer

WITH CTE AS

```
(
    SELECT *,
        RANK() OVER(ORDER BY Salary DESC) AS Rnk
    FROM Employee
)
SELECT *
FROM CTE
WHERE Rnk = 2;
```

DENSE_RANK()

Q1. Assign Dense Rank

Answer

```
SELECT EmpID,  
       EmpName,  
       Salary,  
       DENSE_RANK() OVER(ORDER BY Salary DESC) AS DRank  
FROM Employee;
```

Q2. Find second highest salary

Answer

```
WITH CTE AS  
(  
    SELECT *,  
           DENSE_RANK() OVER(ORDER BY Salary DESC) AS DRank  
    FROM Employee  
)  
SELECT *  
FROM CTE  
WHERE DRank = 2;
```

Q3. Find third highest salary

Answer

```
WITH CTE AS  
(  
    SELECT *,  
           DENSE_RANK() OVER(ORDER BY Salary DESC) AS DRank  
    FROM Employee  
)  
SELECT *  
FROM CTE  
WHERE DRank = 3;
```

NTILE()

Q1. Divide employees into 4 groups

Answer

```
SELECT EmpID,  
       EmpName,  
       Salary,  
       NTILE(4) OVER(ORDER BY Salary DESC) AS Quartile  
FROM Employee;
```

Q2. Find Top 25% employees

Answer

```
WITH CTE AS  
(  
    SELECT *,  
           NTILE(4) OVER(ORDER BY Salary DESC) AS Qtr  
    FROM Employee  
)  
SELECT *  
FROM CTE  
WHERE Qtr = 1;
```

LAG()

Q1. Show previous salary

Answer

```
SELECT EmpID,  
       EmpName,  
       Salary,
```

```
LAG(Salary) OVER(ORDER BY Salary) AS PreviousSalary
FROM Employee;
```

Q2. Show salary difference

Answer

```
SELECT EmpID,
       EmpName,
       Salary,
       Salary -
       LAG(Salary) OVER(ORDER BY Salary)
       AS Difference
FROM Employee;
```

Q3. Compare current sale with previous sale

Answer

```
SELECT SaleID,
       SaleDate,
       Amount,
       LAG(Amount) OVER(ORDER BY SaleDate)
       AS PreviousAmount
FROM Sales;
```

LEAD()

Q1. Show next salary

Answer

```
SELECT EmpID,  
       EmpName,  
       Salary,  
       LEAD(Salary) OVER(ORDER BY Salary)  
       AS NextSalary  
FROM Employee;
```

Q2. Show next sale amount

Answer

```
SELECT SaleID,  
       SaleDate,  
       Amount,  
       LEAD(Amount) OVER(ORDER BY SaleDate)  
       AS NextAmount  
FROM Sales;
```

RUNNING TOTAL

Q1. Running total of sales

Answer

```
SELECT SaleID,  
       SaleDate,  
       Amount,  
       SUM(Amount) OVER  
       (  
         ORDER BY SaleDate  
       ) AS RunningTotal  
FROM Sales;
```


Q2. Department-wise running salary total

Answer

```
SELECT EmpID,  
       DeptID,  
       Salary,  
       SUM(Salary) OVER  
       (  
         PARTITION BY DeptID  
         ORDER BY Salary  
       ) AS RunningTotal  
FROM Employee;
```

CTE

Q1. Employees earning above 70000

Answer

```
WITH EmpCTE AS  
(  
  SELECT *  
  FROM Employee  
  WHERE Salary > 70000  
)  
SELECT *  
FROM EmpCTE;
```

Q2. Department-wise average salary

Answer

```
WITH DeptAvg AS  
(  
  SELECT DeptID,  
         AVG(Salary) AvgSalary  
  FROM Employee
```

```
GROUP BY DeptID
)
SELECT *
FROM DeptAvg;
```

Q3. Find employees earning above department average

Answer

```
WITH DeptAvg AS
(
    SELECT DeptID,
           AVG(Salary) AvgSalary
    FROM Employee
    GROUP BY DeptID
)
SELECT E.*
FROM Employee E
JOIN DeptAvg D
ON E.DeptID = D.DeptID
WHERE E.Salary > D.AvgSalary;
```

VIEW

Q1. Create employee view

Answer

```
CREATE VIEW EmployeeView
AS
SELECT EmpID,
       EmpName,
       Salary
FROM Employee;
```

Q2. Display data from view

Answer

```
SELECT *  
FROM EmployeeView;
```

Q3. Drop view

Answer

```
DROP VIEW EmployeeView;
```

INDEX

Q1. Create index on Salary

Answer

```
CREATE INDEX IX_EmployeeSalary  
ON Employee(Salary);
```

Q2. Create composite index

Answer

```
CREATE INDEX IX_DeptSalary  
ON Employee(DeptID, Salary);
```

Q3. Remove index

Answer

```
SQL Server  
DROP INDEX IX_EmployeeSalary
```

ON Employee;

MySQL

DROP INDEX IX_EmployeeSalary
ON Employee;

STORED PROCEDURE

Q1. Display all employees

Answer

```
CREATE PROCEDURE GetEmployees
AS
BEGIN
    SELECT *
    FROM Employee;
END;
```

Execute:

```
EXEC GetEmployees;
```

Q2. Find employee by ID

Answer

```
CREATE PROCEDURE GetEmployeeById
    @EmpID INT
AS
BEGIN
    SELECT *
    FROM Employee
    WHERE EmpID = @EmpID;
END;
```

Execute:

EXEC GetEmployeeById 101;

TRIGGER

Q1. Audit employee insert

Answer

Audit Table

```
CREATE TABLE EmployeeAudit
(
    AuditID INT IDENTITY(1,1),
    ActionType VARCHAR(50),
    ActionDate DATETIME
);
```

Trigger

```
CREATE TRIGGER trg_InsertEmployee
ON Employee
AFTER INSERT
AS
BEGIN
    INSERT INTO EmployeeAudit
    VALUES
    (
        'Employee Inserted',
        GETDATE()
    );
END;
```

Q2. Audit employee update

Answer

```
CREATE TRIGGER trg_UpdateEmployee
ON Employee
AFTER UPDATE
AS
BEGIN
    INSERT INTO EmployeeAudit
    VALUES
    (
        'Employee Updated',
        GETDATE()
    );
END;
```

Q3. Prevent negative salary

Answer

```
CREATE TRIGGER trg_CheckSalary
ON Employee
INSTEAD OF UPDATE
AS
BEGIN
    IF EXISTS
    (
        SELECT *
        FROM inserted
        WHERE Salary < 0
    )
    BEGIN
        PRINT 'Salary cannot be negative';
        RETURN;
    END

    UPDATE Employee
    SET Salary = i.Salary
```

```
FROM inserted i
WHERE Employee.EmpID = i.EmpID;

END;
```